

TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline – Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data flows, transformation logic, and runtime parameters required to build a production-grade ETL solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytics outputs.

Scope

- In Scope:
 - Technical specifications for AWS Glue Spark jobs
 - Data ingestion patterns from S3 CSV sources
 - PySpark transformation logic for data cleaning and quality
 - Apache Hudi integration for SCD Type 2 implementation
 - AWS Glue Data Catalog integration and table registration
 - Incremental processing logic based on customer data changes
 - Aggregation logic for customer spend analytics
 - S3 storage patterns across raw, curated, and analytics layers
 - Error handling and logging strategies
 - Runtime parameter definitions
- Out of Scope:
 - IAM role and policy definitions
 - Network and VPC configuration
 - AWS Glue job scheduling and orchestration (Step Functions, EventBridge)
 - Data encryption key management
 - Cost optimization strategies
 - Performance tuning beyond standard Spark optimization
 - Athena query optimization
 - Data retention and archival automation

Assumptions

1. AWS Glue job will use Glue 4.0 runtime with Spark 3.3.0 and Python 3.10
 2. Apache Hudi version 0.12.x or compatible is available in Glue environment
 3. CSV files use comma as delimiter and do not contain quoted fields with embedded commas
 4. Date format in Order data is ISO 8601 (YYYY-MM-DD) or can be parsed by Spark's default date parser
 5. PricePerUnit in Order data is a valid decimal number with up to 2 decimal places
 6. Qty in Order data is a valid integer
 7. CustId values in Order data always have corresponding entries in Customer data (referential integrity assumed for inner join)
 8. "Null" string detection is case-sensitive (exact match)
 9. Duplicate detection uses all columns with exact match semantics
 10. Hudi table uses CustId + OrderId as composite record key for SCD2 implementation
 11. Hudi precombine field is OpTs (operation timestamp)
 12. Customer change detection compares Name, EmailId, and Region fields only
 13. Processing timestamp (OpTs) uses UTC timezone
 14. Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` is pre-created
 15. S3 paths are accessible with appropriate IAM permissions attached to Glue job role
 16. Glue job has sufficient DPU allocation (minimum 2 DPUs recommended)
 17. CloudWatch Logs retention is configured at AWS account level
 18. Hudi table type is Copy-on-Write (CoW) for query performance optimization
 19. Parquet compression codec is Snappy for analytics tables
 20. No schema evolution is expected during initial implementation
-

2. Technical Requirements

TR-INGEST-001: Implement Customer Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 source location into a Spark DataFrame with defined schema enforcement and prepare for downstream processing.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

- File Format: CSV
- Delimiter: Comma (,)
- Header: Assumed present (first row)
- Encoding: UTF-8
- Target Details:
- Target Type: In-memory Spark DataFrame
- DataFrame Name: `df\_customer\_raw`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:
 1. Use Spark's `spark.read.csv()` method with schema enforcement
 2. Define explicit StructType schema matching Input Schema
 3. Set options:
 - `header=True`
 - `inferSchema=False` (use explicit schema)
 - `mode=FAILFAST` (fail on malformed records)
 4. Read all CSV files matching the source path pattern
 5. Cache DataFrame if used multiple times downstream
- Validation Rules:
 - Verify source path exists and contains at least one .csv file
 - Validate that all expected columns are present
 - Confirm no schema mismatch errors during read operation
 - Check DataFrame is not empty after read
- Error Handling and Logging:
 - If source path is empty or inaccessible: Log error with path details, raise exception, halt job

- If CSV parsing fails: Log error with file name and line number, raise exception, halt job
- If schema mismatch detected: Log error with expected vs actual schema, raise exception, halt job
- Log successful read with record count to CloudWatch
- Runtime Parameters:
 - `SOURCE\_CUSTOMER\_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/
 - `CSV\_DELIMITER`: ","
 - `CSV\_HEADER`: "true"
- Operational Notes:
 - Ensure Glue job has s3:GetObject permission on source bucket
 - Monitor CloudWatch metrics for read latency and data volume
 - Consider enabling Glue job bookmark if incremental file processing is needed in future
-

TR-INGEST-002: Implement Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order transaction CSV files from S3 source location into a Spark DataFrame with defined schema enforcement and type casting for numeric and date fields.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
 - File Format: CSV
 - Delimiter: Comma (,)
 - Header: Assumed present (first row)
 - Encoding: UTF-8
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - DataFrame Name: `df\_order\_raw`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	

	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Use Spark's ``spark.read.csv()`` method with schema enforcement

2. Define explicit StructType schema:

- OrderId: StringType
- ItemName: StringType
- PricePerUnit: DecimalType(10,2)
- Qty: IntegerType
- Date: DateType
- CustId: StringType

3. Set options:

- ``header=True``
- ``inferSchema=False`` (use explicit schema)
- ``mode=FAILFAST`` (fail on malformed records)
- ``dateFormat=yyyy-MM-dd`` (ISO 8601)

4. Read all CSV files matching the source path pattern

5. Cache DataFrame if used multiple times downstream

- Validation Rules:

- Verify source path exists and contains at least one .csv file
- Validate that all expected columns are present
- Confirm PricePerUnit values are valid decimals
- Confirm Qty values are valid integers
- Confirm Date values are parseable as dates
- Check DataFrame is not empty after read

- Error Handling and Logging:

- If source path is empty or inaccessible: Log error with path details, raise exception, halt job
- If CSV parsing fails: Log error with file name and line number, raise exception, halt job
- If schema mismatch detected: Log error with expected vs actual schema, raise exception, halt job

- If type casting fails (PricePerUnit, Qty, Date): Log error with problematic value, raise exception, halt job
- Log successful read with record count to CloudWatch
- Runtime Parameters:
 - `SOURCE\_ORDER\_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/
 - `CSV\_DELIMITER`: ","
 - `CSV\_HEADER`: "true"
 - `DATE\_FORMAT`: "yyyy-MM-dd"
- Operational Notes:
 - Ensure Glue job has s3:GetObject permission on source bucket
 - Monitor CloudWatch metrics for read latency and data volume
 - Date parsing failures will cause job to fail fast; ensure source data quality
-

TR-CLEAN-001: Implement Data Quality - Null Removal and Deduplication

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Apply data quality rules to remove records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from customer and order datasets.

- Source Details:
 - Source Type: In-memory Spark DataFrames
 - Source DataFrames:
 - `df\_customer\_raw` (from TR-INGEST-001)
 - `df\_order\_raw` (from TR-INGEST-002)
- Target Details:
 - Target Type: In-memory Spark DataFrames
 - Target DataFrames:
 - `df\_customer\_clean`
 - `df\_order\_clean`
- Input Schema:
 - Customer: Same as TR-INGEST-001 Output Schema
 - Order: Same as TR-INGEST-002 Output Schema

- Output Schema:
- Customer: Same as Input Schema (subset of records)
- Order: Same as Input Schema (subset of records)
- Processing / Transformation Logic:
- For Customer Dataset:
 1. Remove rows where any column contains string "Null" (case-sensitive):
 - Use ``filter()`` with condition checking each column `!= "Null"`
 2. Remove rows where any column contains actual NULL:
 - Use ``na.drop(how='any')`` to drop rows with any null values
 3. Remove duplicate rows:
 - Use ``dropDuplicates()`` without specifying columns (considers all columns)
 4. Count removed records:
 - Calculate: ``count_raw - count_clean``
 5. Assign result to ``df_customer_clean``
- For Order Dataset:
 1. Remove rows where any column contains string "Null" (case-sensitive):
 - Use ``filter()`` with condition checking each column `!= "Null"`
 2. Remove rows where any column contains actual NULL:
 - Use ``na.drop(how='any')`` to drop rows with any null values
 3. Remove duplicate rows:
 - Use ``dropDuplicates()`` without specifying columns (considers all columns)
 4. Count removed records:
 - Calculate: ``count_raw - count_clean``
 5. Assign result to ``df_order_clean``
- Validation Rules:
 - Verify cleaned DataFrames contain no "Null" strings in any column
 - Verify cleaned DataFrames contain no NULL values in any column
 - Verify no duplicate rows exist in cleaned DataFrames
 - Confirm at least one record remains after cleaning (log warning if empty)
- Error Handling and Logging:
 - Log count of records removed due to "Null" string for each dataset
 - Log count of records removed due to actual NULL for each dataset
 - Log count of duplicate records removed for each dataset

- Log final record counts for cleaned datasets
- If all records removed: Log warning message, continue with empty DataFrame
- If cleaning operation fails: Log error with exception details, raise exception, halt job
- Runtime Parameters:
- `NULL\_STRING\_VALUE`: "Null"
- `ENABLE\_DEDUPLICATION`: "true"
- Operational Notes:
- String "Null" check is case-sensitive; "null" or "NULL" will not be removed
- Deduplication considers all columns; partial key deduplication not implemented
- Empty DataFrames after cleaning will cause downstream joins to produce empty results
- Monitor CloudWatch logs for data quality metrics
-

TR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer and order datasets to AWS Glue Data Catalog as managed tables, enabling SQL query access via AWS Athena and metadata discovery.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source DataFrames:
- `df\_customer\_clean` (from TR-CLEAN-001)
- `df\_order\_clean` (from TR-CLEAN-001)
- Target Details:
- Customer Table:
- Database: gen_ai_poc_databrickscoe
- Table Name: sdlc_wizard_customer
- Storage Format: Parquet
- Storage Location: Managed by Glue Data Catalog (default warehouse location)
- Write Mode: Overwrite
- Order Table:
- Database: gen_ai_poc_databrickscoe

- Table Name: sdlc_wizard_order
- Storage Format: Parquet
- Storage Location: Managed by Glue Data Catalog (default warehouse location)
- Write Mode: Overwrite

- Input Schema:
- Customer: Same as TR-CLEAN-001 Output Schema
- Order: Same as TR-CLEAN-001 Output Schema

- Output Schema:
- Customer Table Schema: Same as Input Schema
- Order Table Schema: Same as Input Schema

- Processing / Transformation Logic:

- For Customer Table:
 1. Use Spark's `df.write.format("parquet")`` method
 2. Set mode to "overwrite"
 3. Use `saveAsTable()` with fully qualified table name:
``gen_ai_poc_databrickscoe.sdlc_wizard_customer``
 4. Glue Spark session automatically registers table in Data Catalog

- For Order Table:
 1. Use Spark's `df.write.format("parquet")`` method
 2. Set mode to "overwrite"
 3. Use `saveAsTable()` with fully qualified table name:
``gen_ai_poc_databrickscoe.sdlc_wizard_order``
 4. Glue Spark session automatically registers table in Data Catalog

- Validation Rules:
 - Verify database ``gen_ai_poc_databrickscoe`` exists before write operation
 - Confirm table write completes without errors
 - Validate table is queryable via Athena after registration
 - Verify table schema in Data Catalog matches source DataFrame schema

- Error Handling and Logging:
 - If database does not exist: Log error with database name, raise exception, halt job
 - If table write fails: Log error with table name and exception details, raise exception, halt job
 - If Data Catalog registration fails: Log error, raise exception, halt job
 - Log successful table creation with record counts to CloudWatch

- Runtime Parameters:
- `CATALOG\_DATABASE`: gen_ai_poc_databrickscoe
- `CATALOG\_TABLE\_CUSTOMER`: sdlc_wizard_customer
- `CATALOG\_TABLE\_ORDER`: sdlc_wizard_order
- `TABLE\_FORMAT`: parquet
- `WRITE\_MODE`: overwrite
- Operational Notes:
- Ensure Glue job has glue:CreateTable and glue:UpdateTable permissions
- Overwrite mode will replace existing table data on each run
- Parquet format provides columnar storage and compression benefits
- Tables are immediately queryable via Athena after successful write
- Monitor Glue Data Catalog for table metadata updates
-

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Apache Hudi

- Related Functional Requirement ID(s): FR-SCD2-001, FR-INCR-001
- Objective:

Join customer and order data, implement SCD Type 2 logic to track historical changes in customer attributes, and store results in Apache Hudi format with proper versioning and active record management.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source DataFrames:
- `df\_customer\_clean` (from TR-CLEAN-001)
- `df\_order\_clean` (from TR-CLEAN-001)
- Existing Hudi Table: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (for incremental updates)
- Target Details:
- Database: gen_ai_poc_databrickscoe
- Table Name: ordersummary
- Storage Format: Apache Hudi (Copy-on-Write)
- Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Write Mode: Upsert (Hudi)
- Input Schema:

- Customer (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Order (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from Customer.CustId	
	Name	String	Variable	No	Direct map from Customer.Name	
	EmailId	String	Variable	No	Direct map from Customer.EmailId	
	Region	String	Variable	No	Direct map from Customer.Region	
	OrderId	String	Variable	No	Direct map from Order.OrderId	
	ItemName	String	Variable	No	Direct map from Order.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from Order.PricePerUnit	
	Qty	Integer	-	No	Direct map from Order.Qty	
	Date	Date	-	No	Direct map from Order.Date	
	IsActive	Boolean	-	No	Derived: True for current records, False for historical	
	StartDate	Date	-	No	Derived: Current processing date for new/updated records	
	EndDate	Date	-	Yes	Derived: NULL for active records, processing date for closed records	
	OpTs	Timestamp	-	No	Derived: Current processing timestamp (UTC)	

- Processing / Transformation Logic:

- Initial Load (First Run):

1. Perform inner join between `df\_customer\_clean` and `df\_order\_clean` on CustId

2. Add SCD2 columns to joined DataFrame:

- IsActive = True (lit(True))
- StartDate = current_date()
- EndDate = NULL (lit(None).cast(DateType()))
- OpTs = current_timestamp()

3. Write to Hudi table using bulk_insert operation

• Incremental Update (Subsequent Runs):

1. Read existing Hudi table into DataFrame `df\_existing\_ordersummary`

2. Perform inner join between `df\_customer\_clean` and `df\_order\_clean` on CustId to create `df\_new\_joined`

3. Identify changed customer records:

- Compare Name, EmailId, Region between `df\_new\_joined` and `df\_existing\_ordersummary`
- Filter records where any of these fields differ

4. For changed records:

- Close existing active records:
- Filter `df\_existing\_ordersummary` for matching CustId + OrderId where IsActive = True
- Update: IsActive = False, EndDate = current_date(), OpTs = current_timestamp()
- Insert new active records:
- Take changed records from `df\_new\_joined`
- Set: IsActive = True, StartDate = current_date(), EndDate = NULL, OpTs = current_timestamp()

5. For new records (CustId + OrderId not in existing table):

- Set: IsActive = True, StartDate = current_date(), EndDate = NULL, OpTs = current_timestamp()

6. Combine closed records and new/updated active records

7. Write to Hudi table using upsert operation

• Hudi Configuration:

- Table Type: COPY_ON_WRITE
- Record Key: CustId,OrderId (composite key)
- Precombine Field: OpTs
- Partition Path: None (non-partitioned)
- Operation: upsert (for incremental), bulk_insert (for initial load)
- Hudi Write Options:
- `hoodie.table.name`: ordersummary
- `hoodie.datasource.write.recordkey.field`: CustId,OrderId
- `hoodie.datasource.write.precombine.field`: OpTs

- `hoodie.datasource.write.operation`: upsert
- `hoodie.datasource.write.table.type`: COPY_ON_WRITE
- `hoodie.datasource.hive\_sync.enable`: true
- `hoodie.datasource.hive\_sync.database`: gen_ai_poc_databricksco
- `hoodie.datasource.hive\_sync.table`: ordersummary
- `hoodie.datasource.hive\_sync.mode`: hms
- `hoodie.datasource.hive\_sync.partition\_extractor\_class`: org.apache.hudi.hive.NonPartitionedExtractor
- Validation Rules:
 - Verify all active records have IsActive = True and EndDate = NULL
 - Verify all historical records have IsActive = False and EndDate populated
 - Confirm no duplicate active records for same CustId + OrderId combination
 - Validate OpTs is populated for all records
 - Verify Hudi table is queryable via Athena after write
- Error Handling and Logging:
 - If join produces no records: Log warning, create empty Hudi table with schema, continue
 - If Hudi write fails: Log error with exception details, raise exception, halt job
 - If Hive sync fails: Log error, raise exception, halt job
 - If customer has no matching orders (inner join): Exclude from ordersummary, log informational message
 - Log count of new records, updated records, and closed records to CloudWatch
- Runtime Parameters:
 - `HUDI\_TABLE\_NAME`: ordersummary
 - `HUDI\_RECORD\_KEY`: CustId,OrderId
 - `HUDI\_PRECOMBINE\_FIELD`: OpTs
 - `HUDI\_TABLE\_TYPE`: COPY_ON_WRITE
 - `HUDI\_OPERATION`: upsert
 - `TARGET\_ORDERSUMMARY\_PATH`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - `CATALOG\_DATABASE`: gen_ai_poc_databricksco
 - `CATALOG\_TABLE\_ORDERSUMMARY`: ordersummary
- Operational Notes:
 - Ensure Glue job has s3:PutObject permission on target bucket
 - Hudi requires additional Spark configuration and dependencies in Glue job
 - Copy-on-Write table type optimizes for read performance at cost of write performance

- Composite record key (CustId,OrderId) ensures uniqueness at order level
- OpTs as precombine field ensures latest timestamp wins in case of conflicts
- Hive sync enables immediate Athena queryability
- Monitor Hudi commit timeline for compaction and cleaning operations
- Consider enabling Hudi clustering for large tables in future
-

TR-AGG-001: Generate Customer Aggregate Spend Analytics Table

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Calculate total spending per customer per date by aggregating order transaction data and store results in Parquet format for analytical query performance.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source DataFrames:
- `df\_customer\_clean` (from TR-CLEAN-001)
- `df\_order\_clean` (from TR-CLEAN-001)
- Target Details:
- Database: gen_ai_poc_databricksco
- Table Name: customeraggregatespend
- Storage Format: Parquet
- Storage Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Write Mode: Overwrite
- Compression: Snappy
- Input Schema:
- Customer (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Order (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	Name	String	Variable	No	Direct map from Customer.Name	
	TotalAmount	Decimal	20,2	No	Derived: SUM(Order.PricePerUnit × Order.Qty)	
	Date	Date	-	No	Direct map from Order.Date	

• Processing / Transformation Logic:

1. Perform inner join between `df\_customer\_clean` and `df\_order\_clean` on CustId

2. Calculate TotalAmount for each order record:

• Create derived column: `TotalAmount = PricePerUnit Qty`

3. Group by Customer Name and Order Date:

• Use `groupBy("Name", "Date")`

4. Aggregate TotalAmount:

• Use `agg(sum("TotalAmount").alias("TotalAmount"))`

5. Select final columns: Name, TotalAmount, Date

6. Write to S3 in Parquet format:

• Use `df.write.format("parquet")`

• Set mode to "overwrite"

• Set compression to "snappy"

• Set path to `s3://adif-sdlc/analytics/customeragggregatespend/`

7. Register table in Glue Data Catalog:

• Use `saveAsTable()` or manual catalog registration

• Database: gen_ai_poc_databrickscoe

• Table: customeragggregatespend

• Validation Rules:

• Verify TotalAmount is calculated correctly (PricePerUnit × Qty)

- Confirm aggregation produces one row per customer per date
- Validate no NULL values in output columns
- Verify output DataFrame is not empty (log warning if empty)
- Confirm table is queryable via Athena after registration
- Error Handling and Logging:
 - If join produces no records: Log warning, create empty table with schema, continue
 - If aggregation fails: Log error with exception details, raise exception, halt job
 - If write to S3 fails: Log error with path and exception details, raise exception, halt job
 - If catalog registration fails: Log error, raise exception, halt job
 - If customer has no orders (inner join): Exclude from aggregate table, log informational message
 - Log count of aggregated records to CloudWatch
- Runtime Parameters:
 - `TARGET\_AGGREGATE\_PATH`: s3://adif-sdlc/analytics/customeraggregatespend/
 - `CATALOG\_DATABASE`: gen_ai_poc_databrickscoe
 - `CATALOG\_TABLE\_AGGREGATE`: customeraggregatespend
 - `PARQUET\_COMPRESSION`: snappy
 - `WRITE\_MODE`: overwrite
- Operational Notes:
 - Ensure Glue job has s3:PutObject permission on analytics bucket
 - Inner join excludes customers with no orders from aggregate
 - Overwrite mode replaces entire table on each run (full refresh)
 - Snappy compression provides good balance of compression ratio and speed
 - Consider partitioning by Date for large datasets in future
 - Monitor query performance in Athena for optimization opportunities
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: PySpark (Python 3.10)
- Rationale: PySpark provides native integration with AWS Glue, comprehensive DataFrame API, and strong community support for Hudi integration

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)

- Glue Version: 4.0 (Spark 3.3.0, Python 3.10)
- Worker Type: G.1X or G.2X (depending on data volume)
- Number of Workers: Minimum 2, scale based on data volume
- Job Timeout: 2880 minutes (48 hours)
- Max Retries: 0 (fail fast for data quality issues)
- Max Concurrent Runs: 1 (prevent concurrent SCD2 conflicts)

Glue Job Type Expectations

- Standard Glue Job: Suitable for batch processing with predictable data volumes
- Glue Streaming Job: Not applicable (batch processing only)
- Python Shell Job: Not applicable (requires Spark for distributed processing)
- Ray Job: Not applicable (Spark-based processing required)
- Additional Configuration:
 - Enable Glue job bookmarks: No (full refresh pattern for customer/order data)
 - Enable continuous logging: Yes (CloudWatch Logs)
 - Enable Spark UI: Yes (for debugging and performance monitoring)
 - Enable job metrics: Yes (CloudWatch Metrics)
 - Additional Python modules: None required (Hudi included in Glue 4.0)
 - Additional JARs: None required (Hudi included in Glue 4.0)
 - Script location: s3:///scripts/customer_order_pipeline.py
 -

4. Data Design Details

Source Paths / Source Systems

	Source Entity	Source Type	Source Path	Format	Schema Defined In	
	-----	-----	-----	-----	-----	
	Customer Data	S3 CSV	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001	
	Order Data	S3 CSV	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002	

- Note: These are the ORIGINAL and ONLY source systems. No intermediate staging tables or derived sources are introduced.

Target Paths / Target Tables

Target Entity	Target Type	Target Path	Format	Write Mode	Catalog Registration
-----	-----	-----	-----	-----	-----
adif_wizard_customer	Glue Catalog Table	Managed by Glue	Parquet	Overwrite	Yes

sdlc_wizard_order	Glue Catalog Table	Managed by Glue	Parquet	Overwrite	Yes
ordersummary	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi CoW	Upsert	Yes
customeraggregatespend	Glue Catalog Table	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	Overwrite	Yes

Catalog Registration Expectations

- Database: gen_ai_poc_databricksco (pre-existing)
- Tables:
 1. sdlc_wizard_customer
 - Registration Method: Automatic via `saveAsTable()`
 - Storage: Managed table (Glue warehouse location)
 - Queryable: Yes (Athena)
 2. sdlc_wizard_order
 - Registration Method: Automatic via `saveAsTable()`
 - Storage: Managed table (Glue warehouse location)
 - Queryable: Yes (Athena)
 3. ordersummary
 - Registration Method: Automatic via Hudi Hive sync
 - Storage: External table (s3://adif-sdlc/curated/sdlc_wizard/ordersummary/)
 - Queryable: Yes (Athena with Hudi support)
 4. customeraggregatespend
 - Registration Method: Automatic via `saveAsTable()` or manual registration
 - Storage: External table (s3://adif-sdlc/analytics/customeraggregatespend/)
 - Queryable: Yes (Athena)

Partitioning Expectations

- sdlc_wizard_customer: Non-partitioned
- sdlc_wizard_order: Non-partitioned
- ordersummary: Non-partitioned (Hudi table)
- customeraggregatespend: Non-partitioned (consider partitioning by Date in future for large datasets)

File Formats / Table Formats

Entity	File Format	Compression	Table Format	Rationale	
-----	-----	-----	-----	-----	
sdlc_wizard_customer	Parquet	Snappy	Standard	Columnar storage, good compression	

	sdlc_wizard_order	Parquet	Snappy	Standard	Columnar storage, good compression	
	ordersummary	Parquet	Snappy	Hudi CoW	SCD2 support, ACID transactions, time travel	
	customeraggregatespend	Parquet	Snappy	Standard	Columnar storage, optimized for analytics	

Update / Overwrite / Append Behavior

Entity	Write Behavior	Rationale	
-----	-----	-----	
sdlc_wizard_customer	Overwrite	Full refresh on each run	
sdlc_wizard_order	Overwrite	Full refresh on each run	
ordersummary	Upsert (Hudi)	SCD2 requires merge logic, incremental updates	
customeraggregatespend	Overwrite	Full refresh on each run, aggregation recalculated	

- Important Notes:
- No staging tables or intermediate layers are introduced
- All source-to-target mappings use the original sources defined in FRD
- Hudi upsert operation handles both inserts and updates based on record key
- Overwrite mode for catalog tables ensures data consistency but requires full reprocessing
-

5. Processing Details

Data Quality Rules

- Null Handling:
 - Remove records containing string "Null" (case-sensitive, exact match)
 - Remove records containing actual NULL values in any column
 - Apply to both customer and order datasets before downstream processing
 - No null imputation or default value substitution
- Validation Rules:
 - Customer.CustId: Must be non-null, non-empty string
 - Customer.Name: Must be non-null, non-empty string
 - Customer.EmailId: Must be non-null, non-empty string (no email format validation)
 - Customer.Region: Must be non-null, non-empty string
 - Order.OrderId: Must be non-null, non-empty string
 - Order.ItemName: Must be non-null, non-empty string
 - Order.PricePerUnit: Must be non-null, valid decimal, ≥ 0
 - Order.Qty: Must be non-null, valid integer, > 0

- Order.Date: Must be non-null, valid date
- Order.CustId: Must be non-null, non-empty string

Deduplication Rules

- Scope: Apply to both customer and order datasets after null removal
- Method: Remove exact duplicate rows based on all columns
- Logic: Use Spark's `dropDuplicates()` without specifying columns
- Behavior: Keep first occurrence, remove subsequent duplicates
- No partial key deduplication: All columns must match for duplicate detection

Join Rules

- Customer-Order Join (for ordersummary):
 - Join Type: Inner join
 - Join Key: CustId
 - Behavior: Exclude customers with no orders, exclude orders with no matching customer
 - Null Handling: Join keys are guaranteed non-null after data quality processing
- Customer-Order Join (for customeraggregatespend):
 - Join Type: Inner join
 - Join Key: CustId
 - Behavior: Exclude customers with no orders, exclude orders with no matching customer
 - Null Handling: Join keys are guaranteed non-null after data quality processing

Aggregation Rules

- Customer Aggregate Spend:
 - Grouping Columns: Customer.Name, Order.Date
 - Aggregation Function: $\text{SUM}(\text{Order.PricePerUnit} \times \text{Order.Qty})$
 - Output Column: TotalAmount (Decimal 20,2)
 - Null Handling: No nulls expected after data quality processing
 - Zero Handling: Include zero-amount aggregates if they occur

SCD / History Tracking Rules

- SCD Type 2 Implementation for ordersummary:
 - Change Detection:
 - Compare Customer attributes: Name, EmailId, Region
 - Ignore changes in Order attributes (OrderId, ItemName, PricePerUnit, Qty, Date)

- Change detected if any customer attribute differs from existing active record
- Active Record Management:
 - IsActive = True: Current, active record
 - IsActive = False: Historical, closed record
 - StartDate: Date when record became active (current_date() at processing time)
 - EndDate: Date when record was closed (current_date() at processing time), NULL for active records
 - OpTs: Timestamp of record creation or update (current_timestamp() at processing time, UTC)
- Insert Logic:
 - New records (CustId + OrderId not in existing table):
 - IsActive = True
 - StartDate = current_date()
 - EndDate = NULL
 - OpTs = current_timestamp()
- Update Logic:
 - Changed records (customer attributes differ):
 - Close existing active record:
 - IsActive = False
 - EndDate = current_date()
 - OpTs = current_timestamp()
 - Insert new active record:
 - IsActive = True
 - StartDate = current_date()
 - EndDate = NULL
 - OpTs = current_timestamp()
- Hudi Upsert Behavior:
 - Record Key: CustId + OrderId (composite)
 - Precombine Field: OpTs (latest timestamp wins)
 - Operation: Upsert (merge on record key)
 - Table Type: Copy-on-Write (optimized for reads)

Null Handling

- Pre-Processing: Remove all records with "Null" string or NULL values
- Post-Processing: No nulls expected in cleaned datasets

- SCD2 EndDate: NULL allowed (indicates active record)
- Join Behavior: Inner joins exclude null keys by default
- Aggregation Behavior: No null handling required (nulls removed upstream)

Type Casting Expectations

- CSV to DataFrame:
- CustId: String (no casting)
- Name: String (no casting)
- EmailId: String (no casting)
- Region: String (no casting)
- OrderId: String (no casting)
- ItemName: String (no casting)
- PricePerUnit: String → Decimal(10,2) (explicit cast during CSV read)
- Qty: String → Integer (explicit cast during CSV read)
- Date: String → Date (explicit cast during CSV read with dateFormat)
- Derived Columns:
- TotalAmount: Decimal(10,2) × Integer → Decimal(20,2) (automatic promotion)
- IsActive: Boolean (literal value)
- StartDate: Date (from current_date())
- EndDate: Date or NULL (from current_date() or lit(None).cast(DateType()))
- OpTs: Timestamp (from current_timestamp())
-

6. Traceability Matrix

TRD Requirement ID(s)	Original Source	Technical Source	Description
-----	-----	-----	-----
TR-INGEST-001	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	Customer CSV ingestion
TR-INGEST-002	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	Order CSV ingestion
TR-CLEAN-001	df_customer_raw, df_order_raw	df_customer_raw, df_order_raw	Data quality - null removal a
TR-CATALOG-001	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	Glue Catalog registration
TR-SCD2-001	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	SCD Type 2 implementation
TR-SCD2-001	df_customer_clean, existing ordersummary	df_customer_clean, existing ordersummary	Incremental processing (inte
TR-AGG-001	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	Customer aggregate spend ca

- Source Fidelity Verification:
- All technical sources match original FRD sources exactly

- No intermediate staging tables introduced
- No alternative source systems created
- In-memory DataFrames reference original source data only
-

7. Additional ETL Mapping Requirements

TR-INGEST-001: Customer Data Ingestion - Column Mapping

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	-----	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	CustId	Direct map	
	Name	String	Variable	No	Name	Direct map	
	EmailId	String	Variable	No	EmailId	Direct map	
	Region	String	Variable	No	Region	Direct map	

- Target: df_customer_raw (DataFrame)

•

TR-INGEST-002: Order Data Ingestion - Column Mapping

- Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	Target Column	Transformation	
	-----	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	OrderId	Direct map	
	ItemName	String	Variable	No	ItemName	Direct map	
	PricePerUnit	String	Variable	No	PricePerUnit	Cast to Decimal(10,2)	
	Qty	String	Variable	No	Qty	Cast to Integer	
	Date	String	Variable	No	Date	Cast to Date (format: yyyy-MM-dd)	
	CustId	String	Variable	No	CustId	Direct map	

- Target: df_order_raw (DataFrame)

•

TR-CLEAN-001: Data Quality - Column Mapping

- Source: df_customer_raw, df_order_raw (DataFrames)
- Customer Cleaning:

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	CustId	String	CustId	Filter: != "Null" AND NOT NULL, Deduplicate	
	Name	String	Name	Filter: != "Null" AND NOT NULL, Deduplicate	
	EmailId	String	EmailId	Filter: != "Null" AND NOT NULL, Deduplicate	
	Region	String	Region	Filter: != "Null" AND NOT NULL, Deduplicate	

- Order Cleaning:

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	OrderId	String	OrderId	Filter: != "Null" AND NOT NULL, Deduplicate	
	ItemName	String	ItemName	Filter: != "Null" AND NOT NULL, Deduplicate	
	PricePerUnit	Decimal(10,2)	PricePerUnit	Filter: != "Null" AND NOT NULL, Deduplicate	
	Qty	Integer	Qty	Filter: != "Null" AND NOT NULL, Deduplicate	
	Date	Date	Date	Filter: != "Null" AND NOT NULL, Deduplicate	
	CustId	String	CustId	Filter: != "Null" AND NOT NULL, Deduplicate	

- Target: df_customer_clean, df_order_clean (DataFrames)

•

TR-CATALOG-001: Glue Catalog Registration - Column Mapping

- Source: df_customer_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	CustId	String	CustId	Direct map	
	Name	String	Name	Direct map	
	EmailId	String	EmailId	Direct map	
	Region	String	Region	Direct map	

- Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer (Glue Catalog Table)

•

- Source: df_order_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	OrderId	String	OrderId	Direct map	
	ItemName	String	ItemName	Direct map	
	PricePerUnit	Decimal(10,2)	PricePerUnit	Direct map	

	Qty	Integer	Qty	Direct map	
	Date	Date	Date	Direct map	
	CustId	String	CustId	Direct map	

- Target: gen_ai_poc_databrickscoe.sdlc_wizard_order (Glue Catalog Table)

•

TR-SCD2-001: SCD Type 2 Order Summary - Column Mapping

- Source: df_customer_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	CustId	String	CustId	Direct map	
	Name	String	Name	Direct map	
	EmailId	String	EmailId	Direct map	
	Region	String	Region	Direct map	

- Source: df_order_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	OrderId	String	OrderId	Direct map	
	ItemName	String	ItemName	Direct map	
	PricePerUnit	Decimal(10,2)	PricePerUnit	Direct map	
	Qty	Integer	Qty	Direct map	
	Date	Date	Date	Direct map	
	CustId	String	CustId	Join key (not included in output)	

- Derived Columns:

	Target Column	Data Type	Source	Transformation	
	-----	-----	-----	-----	
	IsActive	Boolean	N/A	Derived: True for active records, False for historical	
	StartDate	Date	N/A	Derived: current_date() at processing time	
	EndDate	Date	N/A	Derived: NULL for active, current_date() for closed	
	OpTs	Timestamp	N/A	Derived: current_timestamp() at processing time (UTC)	

- Target: gen_ai_poc_databrickscoe.ordersummary (Hudi Table)

- Final Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source	
--	-------------	-----------	------------------------	----------	--------	--

	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Customer.CustId	
	Name	String	Variable	No	Customer.Name	
	EmailId	String	Variable	No	Customer.EmailId	
	Region	String	Variable	No	Customer.Region	
	OrderId	String	Variable	No	Order.OrderId	
	ItemName	String	Variable	No	Order.ItemName	
	PricePerUnit	Decimal	10,2	No	Order.PricePerUnit	
	Qty	Integer	-	No	Order.Qty	
	Date	Date	-	No	Order.Date	
	IsActive	Boolean	-	No	Derived	
	StartDate	Date	-	No	Derived	
	EndDate	Date	-	Yes	Derived	
	OpTs	Timestamp	-	No	Derived	

•

TR-AGG-001: Customer Aggregate Spend - Column Mapping

- Source: df_customer_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	Name	String	Name	Direct map (grouping key)	
	CustId	String	N/A	Join key (not included in output)	

- Source: df_order_clean (DataFrame)

	Source Column	Data Type	Target Column	Transformation	
	-----	-----	-----	-----	
	Date	Date	Date	Direct map (grouping key)	
	PricePerUnit	Decimal(10,2)	TotalAmount	Multiply by Qty, then SUM	
	Qty	Integer	TotalAmount	Multiply by PricePerUnit, then SUM	
	CustId	String	N/A	Join key (not included in output)	

- Derived Columns:

	Target Column	Data Type	Source	Transformation	
	-----	-----	-----	-----	
	TotalAmount	Decimal(20,2)	Order.PricePerUnit, Order.Qty	SUM(PricePerUnit × Qty) grouped by Name, Date	

- Target: gen_ai_poc_databrickscoe.customeragggregatespend (Glue Catalog Table)

- Final Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source	
	-----	-----	-----	-----	-----	
	Name	String	Variable	No	Customer.Name	
	TotalAmount	Decimal	20,2	No	Derived: SUM(Order.PricePerUnit × Order.Qty)	
	Date	Date	-	No	Order.Date	

-

Assumptions

1. CSV File Encoding: All CSV files are UTF-8 encoded
2. CSV Delimiter Consistency: All CSV files use comma (,) as delimiter without variation
3. CSV Header Presence: All CSV files contain header row as first line
4. Date Format Consistency: All Date values in Order data follow ISO 8601 format (YYYY-MM-DD)
5. Decimal Precision: PricePerUnit values do not exceed 10 digits total with 2 decimal places
6. Integer Range: Qty values fit within standard 32-bit integer range
7. CustId Immutability: CustId values do not change over time (stable customer identifier)
8. OrderId Uniqueness: OrderId values are globally unique across all order data
9. Referential Integrity: All CustId values in Order data have corresponding entries in Customer data
10. Processing Frequency: Glue job is executed on-demand or scheduled (frequency not specified)
11. Data Volume: Initial data volume is manageable with 2 DPU workers (scale as needed)
12. Timezone: All timestamp operations use UTC timezone
13. Hudi Version: Glue 4.0 includes compatible Hudi version (0.12.x or later)
14. Glue Catalog Database: Database `gen\_ai\_poc\_databrickscoe` exists and is accessible
15. IAM Permissions: Glue job execution role has all required S3, Glue, and CloudWatch permissions
16. S3 Bucket Lifecycle: No S3 lifecycle policies interfere with data access during job execution
17. Concurrent Execution: No concurrent Glue job runs to prevent SCD2 conflicts
18. Schema Stability: No schema changes expected in source CSV files during initial implementation
19. Null Representation: Only "Null" (string) and NULL (actual null) are present; no other null representations
20. Customer Change Detection: Only Name, EmailId, and Region changes trigger SCD2 updates

-

- Document Version: 1.0

- Date: 2025-01-29
- Status: Final